Technical report

Snake, Learned by Playing Itself

Self-play reinforcement learning for multi-agent Battlesnake, from simulator to browser

1 Introduction
2 The Game
3 A Batched Simulator
4 Verifying the Simulator
5 Observations, Rewards and Masks
6 Scripted Opponents and Measurement
7 Learning by Self-Play
8 Results: The Duel
9 Results: The Arena
10 Into the Browser
11 Limitations and Next Steps

October 2026 · Alex Kozik · github.com/LambdaAK/SnakeDQN

At a glance

This report documents a complete reinforcement-learning system for *Battlesnake*, the simultaneous-move, multi-player variant of Snake. Two neural networks were trained from scratch by self-play, with no human data and no hand-written features, and both now run live in a web page. The report covers every layer: the game rules and their batched implementation, how that implementation was verified, the learning algorithm, the experiments, and the browser port.

Headline results

Duel (two snakes, 11×11). After 2.5 hours of self-play on a laptop (33.6M game turns), the network wins **91.4%** of 1,000 games against the strongest hand-written bot (95% CI 89.5–93.0), and **90.2%** against a food-greedy bot. For comparison, the strongest bot beats the greedy one only 74.0% of the time.

Arena (six snakes, 19×19). A second network that sees only a 17×17 window around its own head wins **44.7%** of 300 games played as one snake against *five* flood-fill bots. An even contest would win $1/6 \approx 16.7\%$.

Component	Figure	Where
Simulator throughput	214,000 turns/s	Chapter 3
Automated tests	106	Chapter 4
Duel network	2.04M parameters, 8.2 MB ONNX	Chapter 7
Arena network	2.43M parameters, 9.7 MB ONNX	Chapter 9
Duel win rate vs flood-fill bot	91.4%	Chapter 8
Arena win rate vs 5× flood-fill	44.7%	Chapter 9
Training budget, each network	2.5 h, Apple M3 Max GPU	Chapter 7

How to read this report. Four kinds of box appear throughout, as in the course notes this layout is borrowed from.

Definition

A formal object, rule or quantity.

Key idea

The load-bearing decision or insight.

Algorithm

A procedure, as implemented.

Watch out

A subtlety, a bug found and fixed, or a caveat.

Contents

At a glance **2**

1 Introduction **6**

1.1 Goals . . . 6
1.2 System overview . . . 6
1.3 Contributions . . . 7
1.4 A note on history . . . 7

2 The Game **8**

2.1 The board and the snakes . . . 8
2.2 One turn . . . 9
2.3 Head-to-head collisions . . . 9
2.4 Setup . . . 10
2.5 As a Markov game . . . 10

3 A Batched Simulator **11**

3.1 State as arrays . . . 11
3.2 The step, vectorised . . . 11
3.2.1 Occupancy by `bincount` . . . 12
3.2.2 Head-to-head without loops . . . 12
3.3 Randomness you can replay . . . 12
3.4 The environment wrapper . . . 13
3.5 Performance . . . 13

4 Verifying the Simulator **15**

4.1 Rule tests . . . 15
4.2 An independent reference implementation . . . 15
4.3 Batch consistency . . . 16
4.4 The browser port . . . 16
4.5 Summary . . . 17

5 Observations, Rewards and Masks **18**

5.1 Eight egocentric planes . . . 18

5.2 Walls . . . 19

5.3 The arena window . . . 20

5.4 Rewards . . . 20

5.5 Episode boundaries and padding . . . 21

5.6 The action mask . . . 21

6 Scripted Opponents and Measurement **23**

6.1 Five bots . . . 23

6.2 The baseline ladder . . . 24

6.2.1 Single-snake references . . . 24

6.3 The evaluation harness . . . 25

6.3.1 Confidence intervals . . . 25

7 Learning by Self-Play **26**

7.1 Actor and critic . . . 26

7.2 Advantages . . . 26

7.3 The PPO objective . . . 26

7.4 Self-play . . . 27

7.5 The network . . . 28

7.6 Compute . . . 28

8 Results: The Duel **29**

8.1 Learning curve . . . 29

8.2 Training dynamics . . . 30

8.3 Final evaluation . . . 30

9 Results: The Arena **32**

9.1 Measuring a six-player game . . . 32

9.2 Learning curve . . . 33

9.3 Choosing the checkpoint . . . 33

9.4 Final evaluation . . . 34

9.5 The window generalises . . . 34

10 Into the Browser **35**

10.1 Architecture 35

10.2 Modes 35

10.3 Continuous motion from a turn-based game 36

11 Limitations and Next Steps **38**

11.1 What the results do not show 38

11.2 Next steps 38

A Hyperparameters **40**

B Reproducing the Results **41**

C Repository Map **42**

1

Introduction

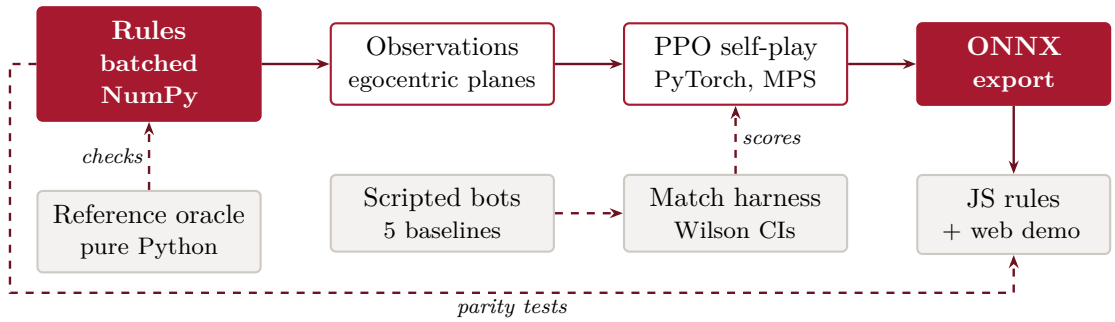
Snake is a game most people learn in an afternoon: steer, eat, grow, and do not run into anything. Battlesnake turns it into a competitive, simultaneous-move game for several players, where the board fills up with bodies and two heads meeting on the same cell settle the matter by length. It is small enough to simulate millions of times an hour, yet rich enough that good play needs planning: cutting opponents off, avoiding traps you cannot see the bottom of, and knowing when to eat and when to fight.

This project builds, from the ground up, an agent that learns this game entirely by playing against copies of itself, and puts the result in a web page where anyone can watch it or play against it.

1.1 Goals

1. **A faithful, fast simulator.** Standard Battlesnake rules, batched so that thousands of games advance in one vectorised step, with correctness established by independent means rather than assumed.
2. **Learning without priors.** No human games, no hand-engineered features in the observation, and a reward that only says who won.
3. **Honest measurement.** Fixed scripted opponents, many games, confidence intervals, and checkpoint selection that does not leak into the reported numbers.
4. **A live demonstration.** The trained networks run client-side in a browser, on a port of the rules that is checked move-for-move against the original.

1.2 System overview



The work is organised in layers, each of which is tested against something independent of itself:

- the **rules** (Chapter 3) against a slow reference implementation and against themselves run one

game at a time (Chapter 4);
- the **observations and rewards** (Chapter 5) against hand-built positions;
- the **agents** against five scripted bots through a common evaluation harness (Chapter 6);
- the **browser port** (Chapter 10) against Python-generated trajectories.

1.3 Contributions

What this report shows

1. A batched Battlesnake engine in pure NumPy reaching 214,000 game turns per second on a laptop CPU, verified exactly against an independent oracle.
2. A duel network that beats the best hand-written heuristic in 91% of games after 2.5 hours of training, learned from a ± 1 win signal.
3. A head-centred *window* observation that decouples the network from board size and player count, enabling a six-snake arena agent that wins nearly three times its fair share against a table of flood-fill bots.
4. A browser deployment with continuous rendering of a turn-based game, and a JavaScript rules port verified against the Python engine.

1.4 A note on history

The repository began as a single-snake Deep Q-Network with a hand-crafted 33-number state: danger flags, A* path quality, dead-end lookahead. That design leaned on features that already encoded much of the answer, ran a path search four times per step, and could not express a second player. The rebuild described here discards those features, replaces the single-player game with the full multi-player rules, and moves from DQN to PPO. Comparing value-based and policy-gradient methods on the new environment is left as future work (Chapter 11).

2

The Game

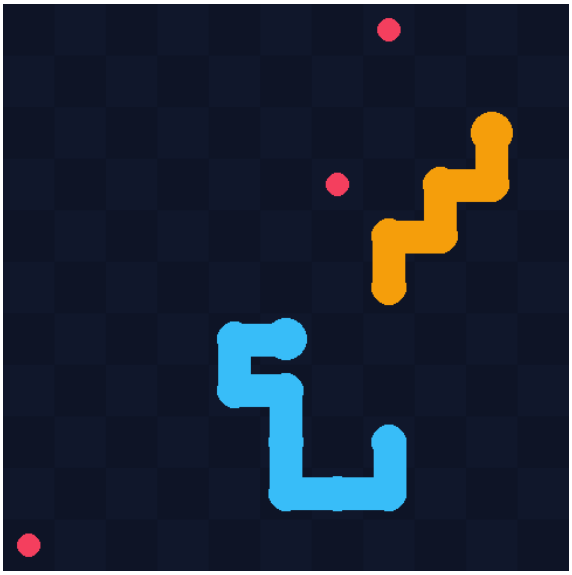


Figure 2.1. A duel at turn 45, both seats played by the trained network. Heads are the larger discs; red dots are food.

2.1 The board and the snakes

Game state

A game is played on a $B \times B$ grid of cells (r, c), row 0 at the top, with $B \geq 7$ ($B = 11$ by default). It has $S \in \{1, \ldots, 8\}$ snakes. Snake k has

- a *body* $(x_0^k, x_1^k, \ldots, x_{L_k-1}^k)$, a list of cells with the head x_0^k first. Cells may repeat; this matters, see below.
- a *length* L_k, the number of entries in the body;
- a *health* $h_k \in \{0, \ldots, h_{\max}\}$ with $h_{\max} = 100$;
- an *alive* flag.

The board also holds a set of food cells F, a turn counter, and the game's *minimum food* $f_{\min}$, which is 1 in standard play.

Actions. Every turn each living snake chooses one of four absolute directions, $a \in \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$, encoded 0–3. All snakes move *simultaneously*; nobody sees the others' choices before committing.

Reversing onto your own neck is legal but fatal, exactly as in the official game.

2.2 One turn

Turn order (Battlesnake standard)

Given every snake's action:

1. **Move.** Each living snake's body becomes $(x_0 + \delta(a),\ x_0,\ \ldots,\ x_{L-2})$: a new head is added and the last segment drops off.
2. **Health.** Every moving snake loses one health point.
3. **Feed.** A snake whose new head is on food resets its health to $h_{\max}$ and *duplicates its last segment*, so its length grows by one. The food is removed after all snakes have fed, so two heads on one food both eat.
4. **Spawn food.** If $|F| < f_{\min}$, or with probability 15% otherwise, one food appears on a uniformly random free cell.
5. **Eliminate.** First remove every snake that starved ($h \le 0$) or left the board. Then, simultaneously among the rest, a snake is eliminated if its head lies on any body segment other than a head (its own included), or if it meets another head whose snake is *at least as long*.
6. **End.** With $S > 1$ the game ends when at most one snake is alive; the survivor, if any, wins. A single snake's game ends when it dies.

Growth is a duplicated tail, not an extra head

When a snake eats, its tail stays put *on the next turn*, because the duplicated segment is what drops off. Two consequences follow, and both are easy to get wrong:

- moving into a cell that an opponent's tail is vacating is *safe*, unless that opponent ate last turn, in which case the tail does not move and the move is fatal;
- a snake's length counts the duplicate. A snake that fills all 64 cells of an 8×8 board by eating the food spawned into its own vacated tail cell ends with length 65, not 64. This was caught by a test (Chapter 6).

2.3 Head-to-head collisions

The head-to-head rule is what makes the game strategic rather than a race to eat. Let P_k be the cell snake k moves into. Snake k is eliminated by a head-to-head if

$$\exists\, j \neq k : \quad P_j = P_k \ \text{ and } \ L_j \geq L_k.$$

Equal lengths eliminate *both* snakes. Being longer therefore makes a snake a threat to every neighbouring head, and this is exactly what one of the observation planes encodes (Chapter 5).

2.4 Setup

Start positions and initial food

Let $a = 1$, $m = \lfloor B/2 \rfloor$, $z = B - 2$. The eight start cells are the four *corners* $\{(a,a),(a,z),(z,a),(z,z)\}$ and the four *edge midpoints* $\{(a,m),(m,a),(m,z),(z,m)\}$. Each game picks one group with probability $\frac{1}{2}$ and fills it first, in random order, so with $S \leq 4$ snakes all start equidistant from the centre. Every snake starts with length 3, all three segments stacked on its start cell.

Food is placed on the centre cell, and for each snake on one of its four diagonal neighbours, chosen uniformly among those that are not a board corner and are not strictly closer to the centre than the snake itself.

2.5 As a Markov game

For learning, the game is a stochastic game with S players. The state is the full board; the transition is the turn order above, whose only randomness is food spawning; and each player's reward is

$$R_k = \begin{cases} +1 & \text{if } k \text{ is the last snake alive,} \\ -1 & \text{on the turn } k \text{ is eliminated, unless every remaining snake falls on that turn,} \\ 0 & \text{otherwise.} \end{cases}$$

A turn on which the last snakes all fall together is a *draw*, worth 0 to each of them. Each player observes the board from its own point of view (Chapter 5). Because every seat runs the same rules and sees an egocentric view, a single policy network can play every seat at once, which is what self-play exploits.

3

A Batched Simulator

Reinforcement learning is hungry for experience: the duel network in this report saw 36 million game turns. A Python loop over games would make that a week; the simulator instead advances N games in a single vectorised step, with no Python loop over games anywhere on the training path.

3.1 State as arrays

Batched game state

All games in a batch share the board size B and snake count S. The state is a handful of NumPy arrays indexed `[env, snake, ...]`:

Field	Shape, dtype	Meaning
`body`	$(N, S, B^2{+}1, 2)$, int16	segment cells, head at index 0
`length`	(N, S), int16	number of valid segments
`health`	(N, S), int16	health points
`alive`	(N, S), bool	snake is in play
`food`	(N, B, B), bool	food cells
`turn`	(N), int32	turns played
`over`	(N), bool	game finished, awaiting reset

Segments past `length` are garbage and never read. The extra slot in $B^2{+}1$ holds the duplicated tail of a snake that has filled the board.

Why a fixed-capacity body array

Storing every body in a padded array of capacity $B^2{+}1$ turns "move every snake" into one slice assignment, `body[m, 1:L] = body[m, 0:L-1]`, over the boolean mask m of moving snakes, with L the longest current length plus one. Short games pay only for their actual length, and no per-snake Python list ever exists.

3.2 The step, vectorised

Each stage of the turn order (Chapter 2) maps to a few array operations.

`step_games(state, actions, spawn_u)`

1. **Validate** that `actions` is an integer array of shape (N, S) with values in 0..3; anything else raises, rather than silently wrapping.
2. **Move** the masked snakes by slicing, write the new heads, and decrement health.
3. **Feed** by looking up `food[env, row, col]` at every new head (clipped for off-board heads, which are masked out). Eaters duplicate their tail with one fancy assignment, reset health, and clear the food.
4. **Spawn** at most one food per game, driven by the externally supplied uniforms `spawn_u` (next section).
5. **Eliminate**: compute survivors of starvation and walls; build an occupancy count of their non-head segments; look up each head in it; resolve head-to-heads with an $S \times S$ comparison of head cells and lengths.
6. **Finish**: advance turns, flag finished games, and report per-snake `ate`, `died` and `alive_before`, plus per-game `game_over` and `winner`.

3.2.1 Occupancy by `bincount`

Several stages ask the same question: how many segments of which snakes sit on each cell? Flattening every cell of every game to one index

$$\iota(n, r, c) = (n \cdot B + r) \cdot B + c,$$

the answer for any mask of snakes and range of segment indices is a single `np.bincount` over the valid segments, reshaped to (N, B^2). Body collisions use segments $1..L{-}1$ of survivors; the action mask uses segments $0..L{-}2$ of living snakes, since tails move away; food spawning uses all segments.

3.2.2 Head-to-head without loops

With cell_k the flattened new head of snake k and surv_k its survivor flag, snake k loses a head-to-head if

$$\bigvee_{j \neq k} [\text{cell}_j = \text{cell}_k] \wedge \text{surv}_j \wedge \text{surv}_k \wedge [L_j \geq L_k],$$

an (N, S, S) boolean tensor reduced over j. With $S \leq 8$ this is cheaper than any alternative.

3.3 Randomness you can replay

Injected uniforms

The step function takes its randomness as an argument: two uniforms per game, $u_0, u_1 \sim \mathcal{U}[0,1)$. A food spawns if $|F| < f_{\min}$ or $u_0 < 0.15$, on the $\lfloor u_1 \cdot n_{\text{free}} \rfloor$-th free cell in row-major order. Given the same uniforms, any implementation must place food on exactly the same cell. This is what lets the batched engine, a slow reference engine, and a JavaScript port be compared *exactly*, step for step (Chapter 4).

Choosing the k-th free cell is itself vectorised: with `free` an (n, B^2) boolean array, the cell is `argmax(cumsum(free, axis=1) > k)`.

3.4 The environment wrapper

`SnakeEnv` wraps the rules in the usual interface. Its constructor takes the number of games, the number of snakes, the board size, a reward name, the maximum health, a seed and the minimum food:

```
env = SnakeEnv(num_envs=1024, num_snakes=2)        # 11x11 Battlesnake
obs, info = env.reset()                            # obs: (1024, 2, 8, 11, 11)
obs, rewards, dones, info = env.step(actions)      # actions: (1024, 2) in {0,1,2,3}
```

Finished games **reset automatically** inside `step`. The returned `obs` is the first frame of the new game, while `info` carries `final_obs`, `final_length`, `final_turn`, `winner` and `game_over` for the game that just ended. It also exposes the `action_mask` and the `active` mask, which Chapter 5 explains.

3.5 Performance

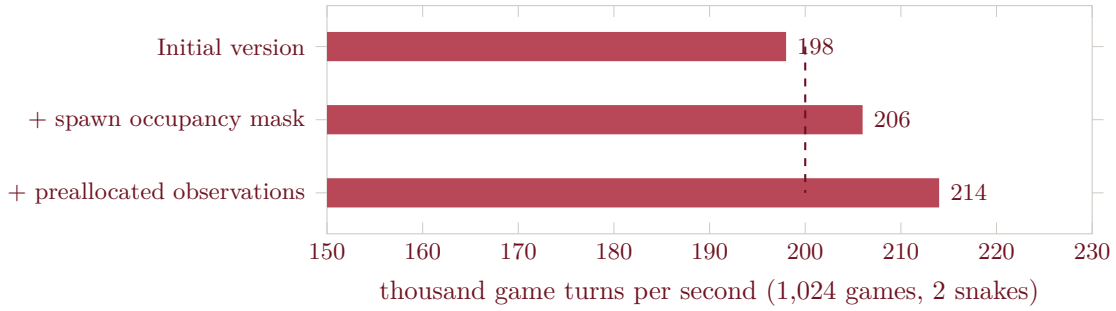


Figure 3.1. Simulator throughput on an Apple M3 Max CPU (medians of repeated runs; the machine varies by about 5% between runs). The dashed line is the 200,000 target.

The target was 200,000 turns per second. Profiling the first version showed time split between building observations (`np.stack`, 18%), occupancy counting (17%) and per-snake layer drawing (14%). Two changes cleared the target without touching behaviour:

- **Spawn occupancy only where needed.** About 15% of games spawn food on a given turn; counting occupancy only for those games removes most of that work.

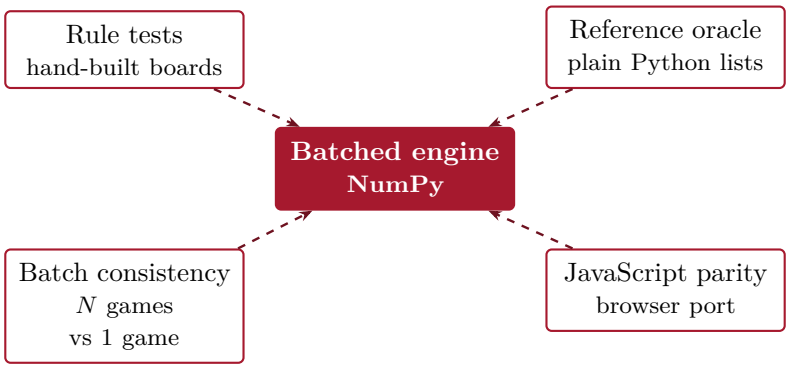
- **Preallocated observations.** Writing each channel into one preallocated array instead of stacking eight temporaries.

A direct comparison of the old and new code over 300 turns of 256 games, in four configurations up to eight snakes on 19×19, produced identical states and observations at every step.

4

Verifying the Simulator

A fast simulator with a subtle bug is worse than a slow correct one: the agent will learn to exploit the bug, and every result built on top inherits it. The engine is therefore checked four independent ways, and all 106 automated tests run on every push.



4.1 Rule tests

Each rule has a test on a hand-built board: walls, self-collision, neck reversal, moving into a vacating tail (safe), moving into a tail that just ate (fatal), eating, starving, eating on the last health point (survives), head-to-head with longer, shorter and equal opponents, collisions with another snake's body and its vacating tail, actions for dead snakes and finished games (ignored), forced and random food spawns, a completely full board (no spawn, no crash), and malformed actions (rejected).

4.2 An independent reference implementation

> **Reference oracle**
>
> A second implementation of the rules for a single game, written for readability: bodies are Python lists, collisions use a `Counter`, free cells are enumerated in row-major order. It shares no code with the batched engine.

Oracle comparison

For many random games (mostly safe moves, 10% random ones):

1. reset finished games, and snapshot every game into the oracle's representation;
2. draw actions and spawn uniforms, and step *both* implementations with them;
3. require every body, health, alive flag, food set, turn counter and game-over flag to match exactly.

The comparison runs on four standard configurations (7×7 solo; 11×11 with 2 and 4 snakes; 8×8 with 3), on an *arena* configuration (19×19, 8 snakes, $f_{\min} = 6$), and on a *starvation* configuration.

A test that passes can still be empty

The first oracle test passed, but replaying its seeds showed 1,753 deaths and *zero* starvations: games end by collision long before 100 health runs out. The starvation rule, and its interaction with eating on the last health point, had never been compared. A dedicated test with $h_{\max} = 10$ now runs the comparison and also asserts that starvations occurred: 300, 559 and 669 of them across three configurations.

4.3 Batch consistency

Stepping 256 games together must give exactly the same result as stepping each of them alone with the same actions and uniforms. This catches the most common class of vectorisation bug: indexing that accidentally mixes games. It is a real test because a single-game batch uses a different slice width (L depends on the longest snake in the batch).

4.4 The browser port

The JavaScript rules are checked against Python-generated transitions: 6,400 recorded steps across five configurations. For each step the test checks the action mask and the full observation of every snake before the step, and every body, health value, food cell and flag after it. The arena configuration also checks the head-centred window observation. Chapter 10 describes the port.

4.5 Summary

Suite	Tests	What it pins down
Rules: reset and step	37	every rule, on hand-built boards
Reference oracle and batch consistency	9	exact agreement on random play
Action mask	5	safe moves, tails, dead snakes
Observations and rewards	12	channel contents, symmetry, windows
Environment	9	auto-reset, masks, determinism
Bots, ladder and evaluation harness	25	bot behaviour, draw accounting, CIs
Agents and training	4	masking, smoke training run
JavaScript parity	1	6,400 recorded transitions
Package and renderer	4	imports, terminal rendering
Total	**106**	

5

Observations, Rewards and Masks

The network is told almost nothing beyond the board itself. Its input is a stack of grids describing the game from its own point of view. Its reward only says who won. One mask removes moves that would kill it outright. This chapter defines all three.

5.1 Eight egocentric planes

Observation

For viewer k, the observation is a tensor $o^k \in [0,1]^{8\times B\times B}$. "Own" refers to snake k, "enemy" to every other living snake; dead snakes are invisible. With body $x_0, \dots, x_{L-1}$:

	Plane	Value at a cell
0	own head	1 at x_0^k
1	own body	1 at $x_1^k, \dots, x_{L_k-1}^k$
2	own age	$(L_k - i)/L_k$ at x_i^k: 1 at the head, $1/L_k$ at the tail
3	enemy heads	1 at every x_0^j, $j \neq k$
4	enemy bodies	1 at every enemy non-head segment
5	dangerous heads	1 at x_0^j for enemies with $L_j \geq L_k$
6	food	1 at every food cell
7	health	$h_k/h_{\max}$ everywhere (0 if k is dead)

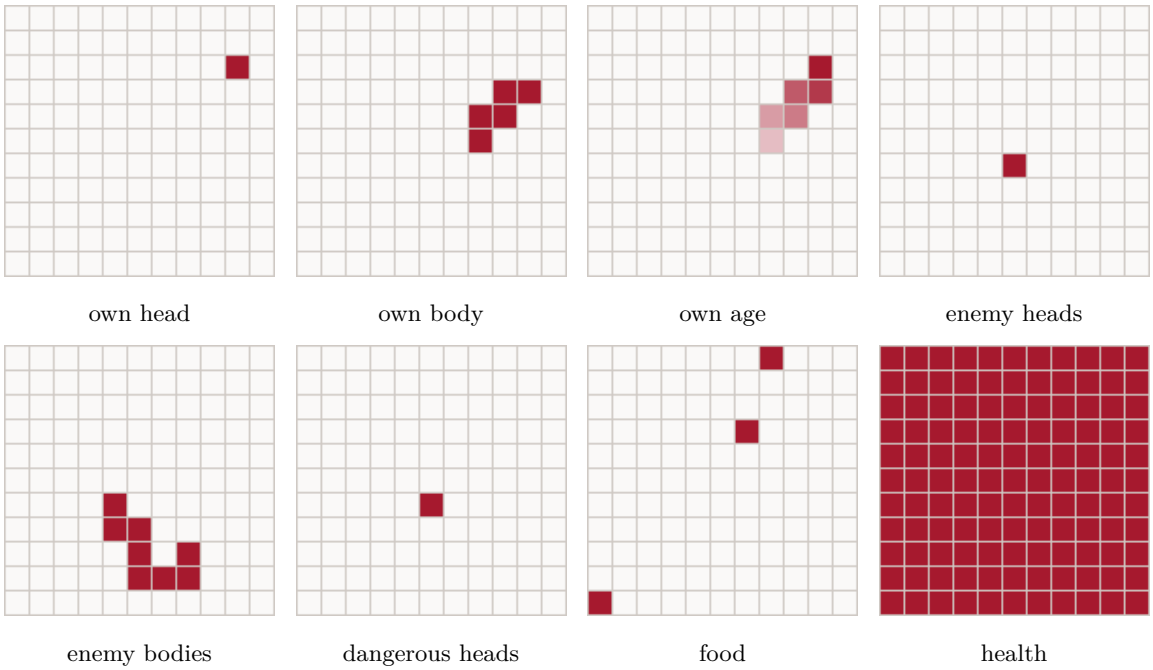


Figure 5.1. The eight planes for the orange snake in the duel position of Figure 2.1. Shade is the plane's value: the age plane fades from head to tail, and the health plane is uniform.

Egocentric, not hand-engineered

Every plane is a direct transcription of the board; nothing is computed that the rules do not already state. The legacy agent's A* path quality and dead-end lookahead are gone. Three design choices still matter:

- **Self versus others** rather than "snake 0 versus snake 1". Swapping two snakes swaps their observations, which is tested, so one network can play any seat.
- **The age plane** tells the network which segment leaves next, which is how it can know a tail cell will be free.
- **The dangerous-heads plane** restates the head-to-head rule (Chapter 2) in a form a convolution can read locally.

Segments that share a cell

At the start all three segments of a snake share one cell, and after eating the tail is duplicated. Planes are written tail-first, so where segments overlap the one nearest the head wins. The duplicated tail cell therefore reads $2/L$ rather than $1/L$ on the turn after eating, a deliberate and documented simplification.

5.2 Walls

Nothing in the eight planes marks the edge of the board. The network supplies it itself: it appends a constant plane of ones before the first convolution. Zero padding then makes that plane drop to

0 exactly outside the board, so every filter that touches the edge can tell. This avoids an extra input plane that would be identical in every position.

5.3 The arena window

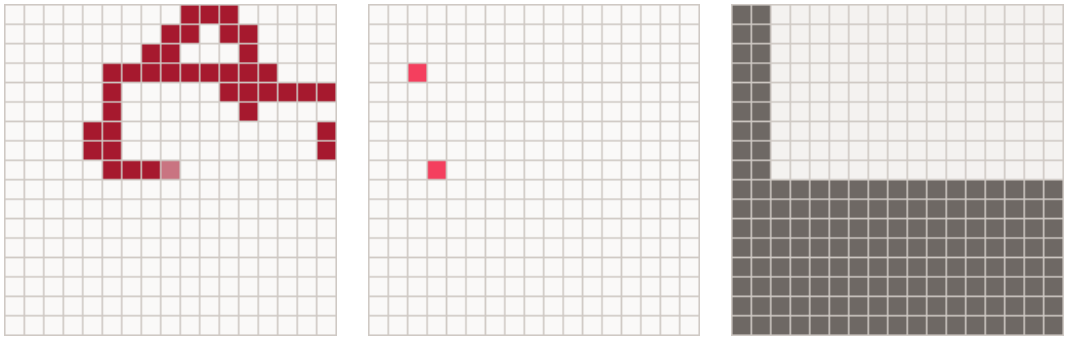
A network whose first fully connected layer expects $64 \cdot 11 \cdot 11$ inputs is bound to an 11×11 board. The arena network instead reads a fixed-size window around its own head.

Window observation

For odd W and viewer k with head at (r_0, c_0), the window observation is $w^k \in [0,1]^{9 \times W \times W}$ with

$$w^k_c(i,j) = \begin{cases} o^k_c(r_0 - \lfloor W/2 \rfloor + i,\ c_0 - \lfloor W/2 \rfloor + j) & \text{if that cell is on the board,} \\ 0 & \text{otherwise,} \end{cases}$$

for the eight planes $c = 0, \ldots, 7$, plus a ninth *wall* plane equal to 1 exactly where the window extends past the board. The arena uses $W = 17$ on a 19×19 board.



snakes (heads lighter) food wall plane

Figure 5.2. The 17×17 window of one arena snake at turn 120, centred on its head. The grey wall plane marks the part of the window that hangs off the board.

Because the window always sits on the snake's own head, its centre is always "me". The same weights serve any board size and any number of opponents. The crop is fully vectorised: row and column index grids are built from the head positions, clipped to the board for the lookup, and multiplied by an in-bounds mask.

5.4 Rewards

Reward functions

`winloss` (default for $S > 1$): $+1$ to the winner when the game ends, -1 to a snake on the turn it dies, and 0 to snakes that die together on the final turn of a game with no winner.

`food` (default for $S = 1$): $+1$ per food eaten, -1 on death.

Training reward: `winloss` plus a small shaping term,

$$r_t^k = R_t^k + 0.05 \cdot \mathbf{1}[\, k \text{ ate at } t \,].$$

The shaping bonus is twenty times smaller than a win, small enough that the agent never prefers eating to winning. It is large enough to make early exploration find food before it can find wins.

5.5 Episode boundaries and padding

With two snakes, a death ends the game. With six it does not, and a dead snake's seat keeps producing "transitions" until the game ends. These are padding: its actions are ignored and its rewards are zero.

Dones and the active mask

$$\text{done}_t^k = \text{died}_t^k \ \vee\ \text{game_over}_t, \qquad \text{active}_t^k = \text{alive}_{t-1}^k \wedge \neg\, \text{over}_{t-1}.$$

Every slot of a game that ends receives *done*, so no slot's trajectory runs into the next game. A transition is real only where *active* holds, and the trainer masks everything else out of the loss.

Found in review

An earlier version set *done* only for snakes alive at the end, so with three or more snakes a dead seat never saw an episode boundary, and nothing marked its padding steps. A probe of 64 games with 3 snakes over 400 turns counted 12,682 such zombie transitions. Two-snake games were never affected, but the arena would have been. The fix above landed before any multi-snake training.

5.6 The action mask

Action mask

Let the *blocked* cells be every segment of every living snake except its last, since tails move away. Move a is *safe* for snake k if $x_0^k + \delta(a)$ is on the board and not blocked. A duplicated tail stays blocked automatically, because its copy is the second-to-last segment.

The policy's logits for unsafe moves are set to -10^9 before the softmax, so the network never wastes probability, or gradient, on walking into a wall. If no move is safe, all four are allowed and the snake dies anyway. The mask is deliberately optimistic in two ways that the network must learn to handle itself:

- it assumes enemy tails move, so it cannot know that an enemy is about to eat and keep its tail where it is;
- it ignores head-to-head risk, which the dangerous-heads plane exposes instead.

6

Scripted Opponents and Measurement

Self-play has no fixed point of reference: the opponent improves as fast as the learner. To know whether the learner is actually getting better, it is scored against opponents that never change. This chapter introduces five hand-written bots, ranks them against each other, and describes the harness that turns matches into numbers with error bars.

6.1 Five bots

Every bot, and every learned agent, has the same interface: a function from a batched game state and a seat index to one action per game. The harness cannot tell them apart.

The baseline bots

random Uniform over the four moves, fatal ones included.

safe_random Uniform over the moves the action mask allows.

greedy Among safe moves, the one that most reduces the Manhattan distance to the nearest food.

flood_fill The strongest bot, built as a Battlesnake heuristic should be. See below.

hamiltonian For a single snake on an even board: follow a fixed cycle through every cell. It never collides and eventually fills the board.

Flood-fill bot, per game

1. Start from the safe moves.
2. **Avoid losing head-to-heads.** Drop moves into a cell that an enemy at least as long could also enter next turn, unless that leaves nothing.
3. **Keep room to move.** Label the connected free regions of the board (tails count as free) with `scipy.ndimage.label`. Keep moves whose region has at least as many cells as the snake is long; if none do, keep those with the largest region.
4. **Eat when it matters.** The snake is *hungry* if its health is ≤ 30 or any enemy is at least as long. Hungry: choose by nearest food, then larger region. Otherwise: choose by larger region, then nearest food.
5. Break remaining ties at random.

6.2 The baseline ladder

Each pair of adjacent bots played 500 games on 11×11.

Matchup	Wins	Draws	Losses	Win rate (95% CI)
flood_fill *vs* greedy	370	22	108	74.0% (70.0–77.7)
greedy *vs* safe_random	401	6	93	80.2% (76.5–83.5)
safe_random *vs* random	498	1	1	99.6% (98.6–99.9)

The ordering random < safe_random < greedy < flood_fill is strict and wide, and is enforced by a test that requires every step of the ladder to exceed 60%.

What the ladder says about the game

Greedy is a strong-looking strategy: it grows quickly and wins most head-to-heads because it is usually longer. Flood-fill still beats it three games in four, almost entirely by not trapping itself. Space, not food, is the scarce resource on an 11×11 board with two snakes on it. The learned agent rediscovers this (Chapter 8).

6.2.1 Single-snake references

On its own, the greedy bot scores a mean of 23.7 food per game (95% CI 22.9–24.5; max 57) over 500 games on 11×11 before it traps itself. The Hamiltonian bot is the opposite extreme: on 8×8 with $h_{\max} = 64$, it fills the entire board in every one of 20 games, finishing at length 65 (Chapter 2) with a score of 62.

6.3 The evaluation harness

`run_matches(policies, n_games, seed)`

1. Create one environment per game, $N = n_{\text{games}}$, with policy i in seat i.
2. Step all games together; every policy acts for its seat in every game.
3. Record each environment's *first* finished game only. Later games in an environment that auto-reset are ignored, so short games are not over-counted.
4. Per seat: a *win* if it is the winner; a *draw* if there is no winner and the seat was active on the final turn; otherwise a *loss*.

Counting each game exactly once

Without step 3, a fast-finishing environment would contribute several games while a slow one contributed one, biasing results toward whatever ends games quickly. A deterministic test pins this down. Two snakes that only ever move up die at the top wall at turn $r_0 + 1$, where r_0 is their start row. Every win, draw and loss, and the mean game length, is therefore predictable from the start rows, and the test checks them exactly.

6.3.1 Confidence intervals

Wilson score interval

For w wins in n games, $\hat{p} = w/n$ and $z = 1.96$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}.$$

The Wilson interval behaves well near 0 and 1, where most of the interesting results in this report sit; the naive normal interval would extend past 100% there. For single-snake scores, a 95% percentile bootstrap over 2,000 resamples gives the interval on the mean. Every run is reproducible from one seed: environments, food and bots all draw from seeded generators.

7

Learning by Self-Play

Both networks are trained with Proximal Policy Optimization (PPO), an on-policy actor–critic method, in a self-play loop: the policy being trained controls most of the snakes in every game. This chapter states the objective, the self-play scheme, the network, and the compute.

7.1 Actor and critic

A single network with parameters θ has two heads. Given observation o, it outputs action logits, from which a masked softmax gives the policy $\pi_\theta(a \mid o)$, and a scalar value estimate $V_\theta(o)$ of the return. The value head is what the web page displays as the network's "confidence" (Chapter 10).

7.2 Advantages

Generalized advantage estimation

For one seat's trajectory with rewards r_t and episode boundaries d_t, define the TD residual and the advantage estimate

$$\delta_t = r_t + \gamma\,(1 - d_t)\,V(o_{t+1}) - V(o_t), \qquad \hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \prod_{m=0}^{l-1} (1 - d_{t+m})\;\delta_{t+l},$$

with $\gamma = 0.99$ and $\lambda = 0.95$, computed by one backward pass over the rollout. The value targets are $\hat{R}_t = \hat{A}_t + V(o_t)$. The last step of a rollout bootstraps from the value of the following observation.

The factor $(1 - d_t)$ is where the dones of Chapter 5 matter: since every slot of a finished game is marked done, no advantage ever leaks from one game into the next game played in the same environment slot.

7.3 The PPO objective

Let π_{old} be the policy that collected the rollout and

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\text{old}}(a_t \mid o_t)}.$$

Clipped surrogate and total loss

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\ \mathrm{clip}\big(\rho_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\,\hat{A}_t\Big)\Big], \qquad \epsilon = 0.2,$$

$$\mathcal{L}(\theta) = -L^{\mathrm{CLIP}}(\theta) + c_v\,\mathbb{E}_t\Big[\tfrac{1}{2}\big(V_\theta(o_t) - \hat{R}_t\big)^2\Big] - c_e\,\mathbb{E}_t\big[\mathcal{H}(\pi_\theta(\cdot \mid o_t))\big],$$

with $c_v = 0.5$ and $c_e = 0.01$. Advantages are normalised to zero mean and unit variance over each batch.

Why PPO here

The game is on-policy friendly: experience is cheap (Chapter 3), and the data distribution shifts every iteration anyway because the opponent *is* the policy. PPO's clipping keeps each update close to the policy that gathered the data. That matters doubly in self-play, where a large jump changes both sides of every game at once.

7.4 Self-play

One training iteration

1. With probability 0.3 per environment, the non-learner seats are played by a *frozen snapshot*: one set of past weights, drawn uniformly from a pool for this iteration. Otherwise the learner plays every seat.
2. Run $T = 64$ turns in all environments, sampling actions from the masked policy. Store observations (float16), masks, actions, log-probabilities, values and rewards.
3. Mark a transition *valid* if its seat was played by the learner and was active (Chapter 5). Compute GAE over the full rollout, then keep valid transitions only.
4. Optimise $\mathcal{L}$ for 4 epochs over the valid transitions in minibatches of 4,096, using Adam (learning rate 2.5×10^{-4}, $\epsilon = 10^{-5}$) and gradient norm clipping at 0.5.
5. Every 10 iterations, add the current weights to the snapshot pool (the most recent 20 are kept). Every 25–30 iterations, evaluate against the scripted bots and save a checkpoint.

Two sources of opponents

Playing the current self is the strongest signal, and every seat yields training data, so a two-snake game produces two transitions per turn. Playing frozen past selves guards against *forgetting*: a policy that only ever meets its current self can drift into a strategy that loses to something it used to beat. Seats a snapshot plays generate no training data.

Each environment re-draws its opponent type whenever its game ends. In the duel run, 512 environments $\times$ 64 turns gives 32,768 game turns per iteration and about 55,000 valid learner transitions. The arena run uses 256 environments of six snakes each.

7.5 The network

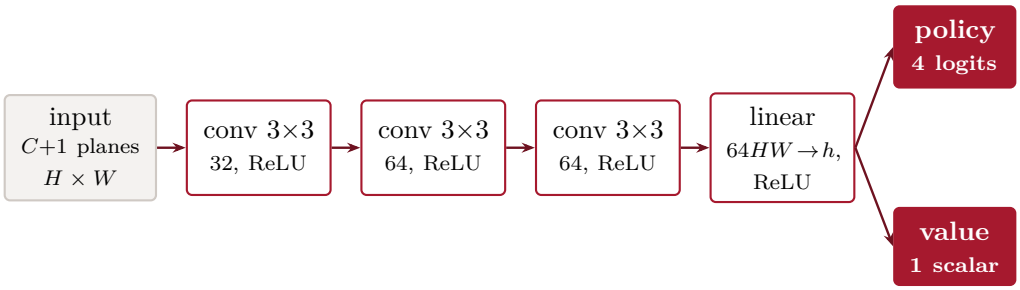


Figure 7.1. The actor–critic. All convolutions use padding 1, so spatial size is preserved; the extra input plane is the constant ones plane that reveals walls through zero padding.

	Input C+1, $H\times W$	**Hidden h**	**Parameters**
Duel network	9, 11×11	256	2,042,053
Arena network	10, 17×17 (window)	128	2,426,597

Almost all parameters sit in the first linear layer, $64HW \cdot h$, which is why the arena network halves h to compensate for its larger window. A fully convolutional design would remove the dependence on $H \times W$ altogether. The window observation achieves the same independence from board size while keeping this simple architecture.

7.6 Compute

Both networks trained for 2.5 hours on an Apple M3 Max, using PyTorch's Metal (MPS) backend for the network and the CPU for the simulator.

	Seconds per iteration	**Relative**
CPU, 12 threads	≈ 70	$1\times$
Apple GPU (MPS)	$\approx$ 5–6	$\approx 10\times$

The GPU mattered more than expected

The network is small, so CPU training looked plausible. In practice the PPO update, four epochs over $\sim$ 55,000 samples through a 2M-parameter network, dominated each iteration, and moving it to the GPU was the difference between a 2.5-hour run and a day-long one. The simulator stayed on the CPU throughout.

8

Results: The Duel

The duel network plays two-snake Battlesnake on the standard 11 × 11 board. It trained for 2.5 hours: 1,109 iterations, 36.3 million game turns.

8.1 Learning curve

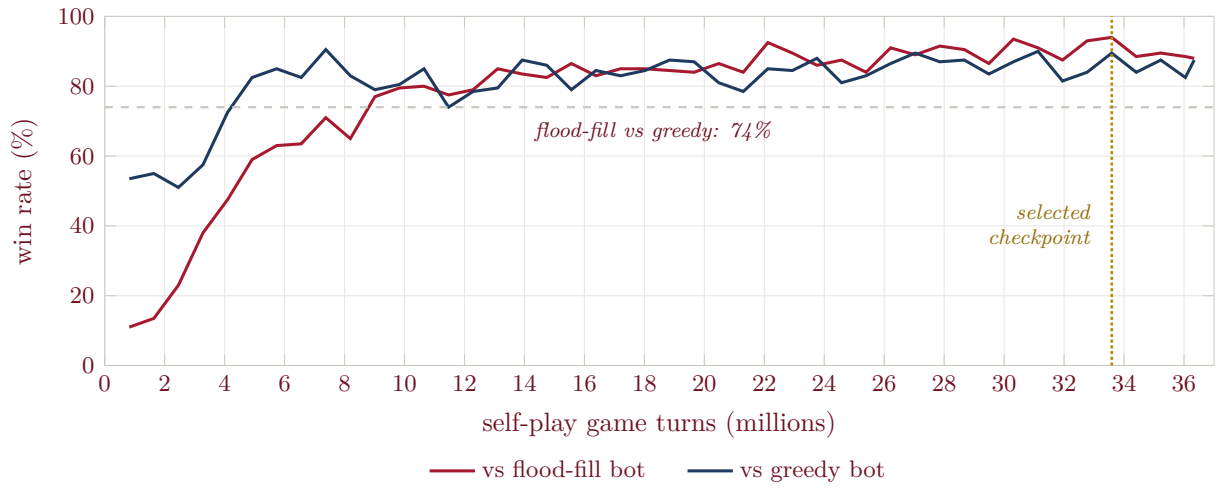


Figure 8.1. Win rate of the duel network during training, measured every 25 iterations over 200 games against each bot (deterministic play). The grey line is the flood-fill bot's own win rate against greedy, for scale.

The curve has three phases.

- **Survival** (0–4M turns). The network first learns not to die. Against the greedy bot it starts at 54% almost immediately, because greedy plays into dead ends that a cautious random walker avoids. Against flood-fill it starts at 11%.
- **Competence** (4–10M). The win rate against flood-fill climbs to about 80% by 10.6M turns. From 9M turns on, the network beats flood-fill more often than flood-fill beats greedy.
- **Refinement** (10–36M). Slow gains to around 90% against both bots, with checkpoint-to-checkpoint noise of a few points, about what 200-game evaluations produce.

8.2 Training dynamics

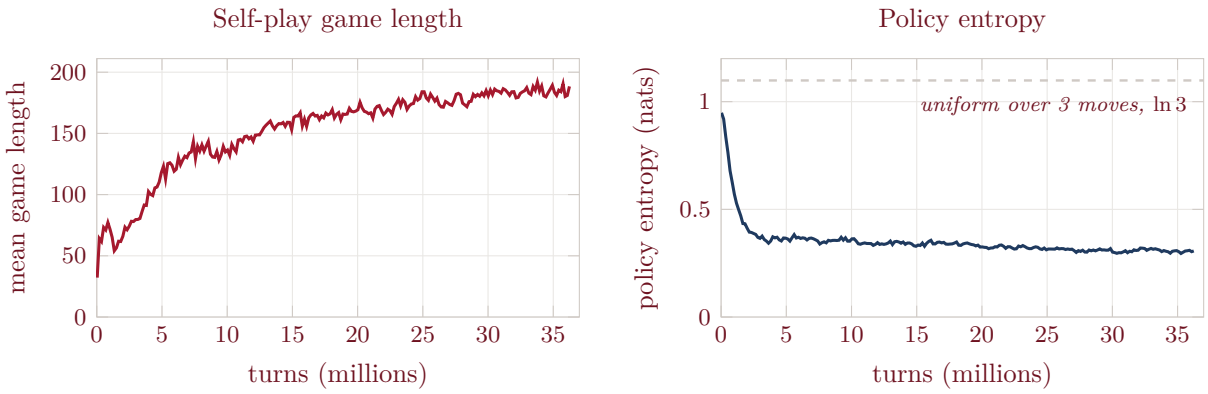


Figure 8.2. Left: mean length of self-play games, which grows from about 30 to about 185 turns as both sides learn to survive. Right: entropy of the masked policy, falling from 0.95 nats (close to uniform over the usual three safe moves) to about 0.3 nats as play becomes decisive.

Self-play games lengthening sixfold is the clearest sign that both copies of the network are improving together: neither side finds easy kills against itself any more. Entropy falls steadily but never collapses. The 0.01 entropy bonus keeps enough randomness that exploration continues to the end of the run.

8.3 Final evaluation

The checkpoint with the best score (win rate $+\frac{1}{2}$ draw rate against flood-fill) was iteration 1,025, at 33.6M turns. It was then evaluated on 1,000 *fresh* games against each bot, with a seed never used during training or selection.

Opponent	W	D	L	Win rate (95% CI)	Game length	Final length
					turns	net / bot
flood_fill	914	4	82	**91.4%** (89.5–93.0)	99.3	13.1 / 12.5
greedy	902	64	34	**90.2%** (88.2–91.9)	63.1	9.4 / 10.5
safe_random	994	2	4	**99.4%** (98.7–99.7)	36.7	8.3 / 4.0

It wins without out-eating its opponent

Against the greedy bot the network wins 90% of games while ending them *shorter* than its opponent, at 9.4 segments against 10.5. It is not winning the race for food. It wins by position: cutting off space, steering the opponent toward walls and its own body, and declining head-to-heads it would lose. These are exactly the ideas built into the flood-fill bot by hand, and that bot also beats greedy. The network found them from a ± 1 signal, and against flood-fill itself it wins 91% of games.

Draws against greedy

6.4% of games against greedy are draws, against 0.4% for flood-fill. Greedy charges straight at food; when both snakes are equally long and arrive at the same cell, both die. The network does not always avoid this; it treats a draw as better than the risk of losing. A reward that penalised draws more than 0 would push it further.

9

Results: The Arena

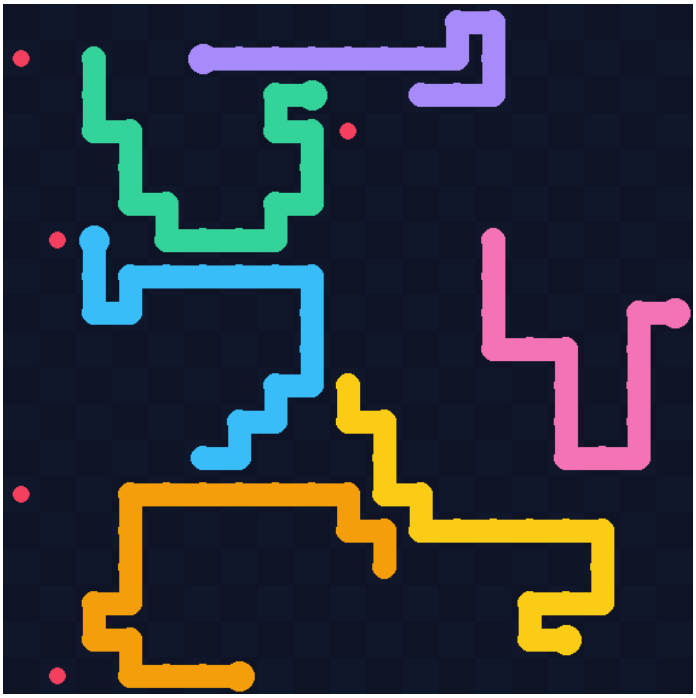


Figure 9.1. An arena game at turn 120: six copies of the arena network on a 19×19 board, all six still alive.

The arena is a free-for-all: six snakes on a 19×19 board, with at least six food on the board at all times, so a crowded board does not simply starve. The arena network reads the 17×17 window of Section 5.3, so the same weights also play the browser's arena with anywhere from 2 to 8 snakes.

9.1 Measuring a six-player game

> **One against five**
>
> A six-player game cannot be scored as a single head-to-head. Instead the network takes one seat and the other *five* are all the same scripted bot. If the network were exactly as good as the bot, it would win $1/6 \approx 16.7\%$ of games. Every arena win rate should be read against that fair share, not against 50%.

9.2 Learning curve

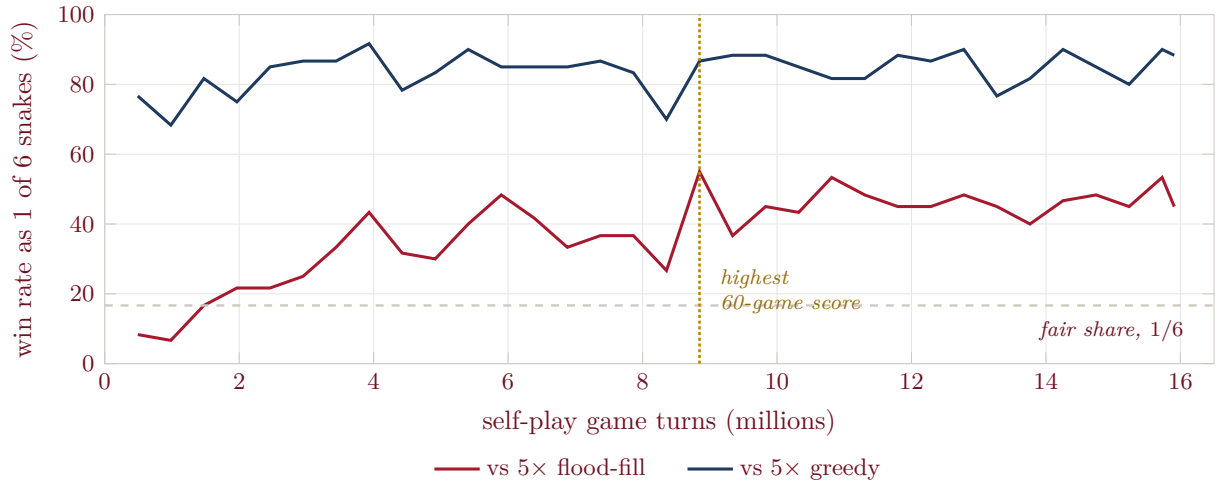


Figure 9.2. Arena win rate during training, evaluated every 30 iterations over 60 games. The dotted line marks the checkpoint that scored highest on these small evaluations; it was not the one finally used (see below).

Against five greedy bots the network is ahead of its fair share from the very first checkpoint, at 77% after half a million turns. Greedy bots in a crowd mostly kill each other, and a snake that merely avoids them survives. Against five flood-fill bots it starts below fair share, at 8%, and passes 40% within 4M turns.



Figure 9.3. Mean length of six-snake self-play games, growing from about 50 turns to over 1,000 as the snakes learn to share the board without colliding.

9.3 Choosing the checkpoint

The "best" checkpoint was not the best

Checkpoints were scored during training on only 60 games each. Picking the maximum of 33 noisy scores favours a checkpoint that got lucky, a selection bias. The highest scorer was iteration 540, at 8.8M turns, yet later checkpoints looked at least as strong. Two candidates were therefore compared on a separate seed, 777, never used for the final numbers:

200 games, seed 777	**vs 5× flood-fill**	**vs 5× greedy**
iteration 540 (highest 60-game score)	35.0%	81.0%
iteration 971 (end of training)	**47.5%**	**92.5%**

The final checkpoint was chosen. The reported results below come from a third seed, 12345, so neither the training-time scores nor this comparison inflates them.

9.4 Final evaluation

Opponents (×5)	**W**	**D**	**L**	**Win rate (95% CI)**	**Game length** turns	**Final length** net / bots
flood_fill	134	2	164	**44.7%** (39.1–50.3)	385.2	43.0 / 25.1
greedy	264	2	34	**88.0%** (83.8–91.2)	163.8	24.7 / 14.3
safe_random	296	0	4	**98.7%** (96.6–99.5)	205.7	32.4 / 5.3

300 games per row; fair share 16.7%.

Nearly three times its fair share

Against a table of five flood-fill bots, each built to protect its own space, the network wins 44.7% of six-player games, 2.7× the 16.7% an equal player would get. It also grows much longer than its opponents, 43 segments against 25. In a crowded game, length is both a weapon (it wins head-to-heads) and a wall that takes space from everyone else, and the network uses it as both.

9.5 The window generalises

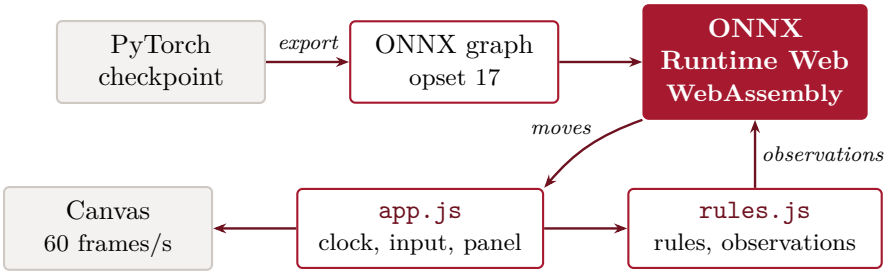
Because the network never sees the whole board, nothing in it depends on the board size or the number of snakes. In the browser it plays two to eight snakes on 19×19 without retraining. No numbers are reported for configurations other than the six-snake training setup, so how well this generalisation holds up is unmeasured.

10

Into the Browser

The final layer puts both networks on a static web page. There is no server: the game rules, the networks and the rendering all run in the visitor's browser.

10.1 Architecture



- **Export.** Each checkpoint is exported to ONNX (opset 17) with a dynamic batch dimension. ONNX Runtime then runs the export on random inputs, and the export script requires its logits and values to match PyTorch to within a relative 10^{-3}.
- **Inference.** ONNX Runtime Web 1.20.1 runs on WebAssembly with a single thread, which avoids the cross-origin isolation headers that multi-threading would require of the host. The duel network (8.2 MB) loads with the page; the arena network (9.7 MB) loads the first time Arena mode is opened.
- **Rules.** `rules.js` is a single-game port of the engine, including the eight-plane and window observations, and is checked against the Python engine on 6,400 transitions (Chapter 4). What the page shows is the game the networks were trained on, not an approximation of it.

10.2 Modes

Mode	Board	Seats
AI vs AI	11×11	duel network in both seats, sampling from its policy
You vs AI	11×11	duel network (most likely move) against the player
AI vs bot	11×11	duel network against the greedy bot
Arena	19×19	2–8 copies of the arena network, sampling; $f_{\min}$ equals the snake count

Two deterministic copies mirror each other

With both seats playing their most likely move, the first AI-vs-AI games ended in identical head-on draws on turn four. The starts are symmetric, so two copies of one deterministic policy make symmetric choices into the centre food. The all-AI modes therefore *sample* from the policy, as in training. The modes against a human or a bot play the most likely move, as in evaluation.

A side panel lists the snakes and their lengths, live.

10.3 Continuous motion from a turn-based game

The game advances in whole turns, but the page must look fluid, like a modern Snake game. The fix is to separate logic from drawing: the rules move in discrete steps, and the renderer interpolates between the last two states.

Interpolated body

Let $p_0, \ldots, p_{L'-1}$ be a snake's cells before the latest turn and $c_0, \ldots, c_{L-1}$ after it, so $c_i = p_{i-1}$ for $i \geq 1$. At fraction $t \in [0, 1]$ of the turn, the snake is drawn as the polyline

$$\big((1-t)\, p_0 + t\, c_0,\ \ c_1,\ \ldots,\ c_{L-1},\ \ (1-t)\, p_{L'-1} + t\, c_{L-1} \big)$$

through cell centres, stroked with round joins and caps.

At $t = 1$ the head sits at c_0 and the tail at c_{L-1}. At the start of the next turn, $t = 0$, they sit at $p'_0 = c_0$ and $p'_{L-1} = c_{L-1}$: the drawing is continuous across turn boundaries. When a snake has just eaten, the duplicated tail makes the last two points coincide, so the tail correctly stays put.

Three bugs on the way to fluid

1. **Ease-in, ease-out.** The first version eased each step with a smoothstep curve. Its velocity is zero at both ends of every turn, so the snake visibly stopped at every cell. Linear interpolation moves at constant speed.
2. **Stalls between turns.** Each turn waited for the network's next move before starting. The fix computes the AI's moves for the next turn while the current one animates, and drives the game from the animation loop on a fixed-cadence clock. A frame-by-frame measurement then showed the snake moving in every one of 90 consecutive frames.
3. **Diagonal tails.** The first polyline joined the moving tail straight to c_{L-2}, skipping c_{L-1}, the cell the tail is entering. At a corner that cut the corner diagonally. Including c_{L-1}, as in the definition above, keeps every segment on the grid.

Player input is buffered and read at the last moment before each turn. Arrow keys, WASD, swipes, and clicks on the board, which steer toward the clicked cell, all feed the same queue. A reversal

onto the snake's own neck is ignored rather than allowed to kill it.

11

Limitations and Next Steps

11.1 What the results do not show

Caveats on the numbers

- **One seed per network.** Each network is a single 2.5-hour run. The confidence intervals cover evaluation noise, not training noise. A second seed could land a few points higher or lower.
- **Scripted opponents only.** The win rates are against hand-written bots. They establish that the networks are strong relative to sensible heuristics, not how they rank among competitive Battlesnake agents or human players.
- **Exact replay needs more care.** The simulator and evaluation are seeded and reproducible. Training on the Apple GPU is not bit-for-bit deterministic, so rerunning training reproduces the curves only statistically.
- **Arena generalisation is unmeasured.** The arena network is evaluated only in its training configuration of six snakes. Its play with 2, 3 or 8 snakes in the browser is plausible but untested.
- **Draws are free.** The reward gives a draw 0, the same as a game still in progress. Against greedy this leaves 6.4% of games drawn (Chapter 8).

11.2 Next steps

An algorithm comparison

The environment and harness were built so that algorithms can be compared fairly, and this is the natural next experiment. DQN and its Rainbow components (double Q-learning, dueling heads, prioritised replay, n-step returns), A2C and PPO would each be trained on the same task with the same budget of environment turns, across several seeds, and compared on learning curves and final win rates. Value-based methods face a particular challenge in self-play, where the replay buffer holds games against opponents who no longer exist.

- **A league and Elo ratings.** Rate every checkpoint against every other, so progress is measured within self-play itself and not only against fixed bots.
- **A fully convolutional network.** Removing the large linear layer would make the duel network board-size independent too, and cut its size several-fold.
- **A public ladder.** The rules follow the official game, so the duel network could play live on

the public Battlesnake arena behind a small server.

- **Reward design.** Penalise draws, or rank the five losers in the arena by survival time, so that coming second is worth more than coming last.

A

Hyperparameters

Setting	Duel	Arena
Board, snakes	11×11, 2	19×19, 6
Minimum food $f_{\min}$	1	6
Observation	full board, 8 planes	17×17 window, 9 planes
Hidden width h	256	128
Parallel games N	512	256
Rollout length T	64	64
Discount γ, GAE λ	0.99, 0.95	0.99, 0.95
Clip ϵ	0.2	0.2
Epochs, minibatch	4, 4,096	4, 4,096
Optimiser	Adam, 2.5×10^{-4}, ϵ=10^{-5}	same
Value coef. c_v, entropy coef. c_e	0.5, 0.01	0.5, 0.01
Gradient-norm clip	0.5	0.5
Food bonus	0.05	0.05
Snapshot opponent probability	0.3	0.3
Snapshot pool size, interval	20, every 10 iterations	20, every 10 iterations
Evaluation	every 25 its., 200 games	every 30 its., 60 games
Wall-clock budget	2.5 h	2.5 h
Iterations, game turns	1,109, 36.3M	971, 15.9M
Reported checkpoint	iteration 1,025	iteration 971

B

Reproducing the Results

Everything runs through `uv`. The training extras (PyTorch, ONNX, Pillow) are an optional dependency group.

```
uv run pytest                                    # 106 tests
uv run python scripts/benchmark_env.py           # simulator throughput
uv run python scripts/evaluate.py flood_fill greedy --games 500

uv sync --group train
# duel network
uv run --group train python scripts/train_selfplay.py --device mps --hours 2.5
uv run --group train python scripts/final_eval.py runs/selfplay/best.pt --games 1000
# arena network
uv run --group train python scripts/train_selfplay.py --run-dir runs/arena \
    --board 19 --num-snakes 6 --window 17 --min-food 6 --hidden 128 \
    --num-envs 256 --device mps --hours 2.5
uv run --group train python scripts/final_eval.py runs/arena/latest.pt \
    --board 19 --num-snakes 6 --min-food 6 --games 300 --out results/ppo_arena.json
# browser assets, then serve the demo
uv run --group train python scripts/build_web_assets.py --run-dir runs/selfplay \
    --final-eval results/ppo_selfplay.summary.json
python3 -m http.server --directory web 8000
# this report's figures
uv run --group train python docs/report/make_figures.py
```

All evaluation numbers in this report are stored as JSON in `results/`, alongside the full per-iteration training logs and configurations of both runs.

C

Repository Map

Path	Contents
snake_rl/env/	rules, observations, rewards, batched SnakeEnv, terminal renderer
snake_rl/baselines/	random, safe_random, greedy, flood_fill and hamiltonian bots
snake_rl/agents/	actor–critic network and its harness wrapper
snake_rl/eval/	match runner with Wilson and bootstrap intervals
scripts/	training, evaluation, export, benchmarking, play, GIF rendering
tests/	the 106-test suite, including the reference oracle
web/	the browser demo: rules port, app, models
results/	evaluation JSON and training logs
docs/report/	this report and the script that draws its figures